SINGAPORE UNIVERSITY OF SOCIAL SCIENCES

ICT133

Examination - July Semester 2023

Structured Programming

Thursday, 16 November 2023 01:00 pm - 03:00 pm

Time allowed: 2 hours

INSTRUCTIONS TO STUDENTS:

1. This examination contains **4** questions and comprises **9** printed pages (including cover page).
2. You must answer **4** questions.
3. All answers must be written in the answer book.
4. This is **Closed-book** examination.
5. Refer to Appendix 1 attached to the question paper.
6. You can use ONLY built-in functions and packages covered in the module, e..g. sum, max,min,math,random. You can use ONLY built-in types. Do not write classes.

At the end of the examination.
Please ensure that you have written your examination number on each answer book used.

Failure to do so will mean that your work cannot be identified.
If you have used more than one answer book, please tie them together with the string provided.

THE UNIVERSITY RESERVES THE RIGHT NOT TO MARK YOUR SCRIPT IF YOU FAIL TO FOLLOW THESE INSTRUCTIONS.

ICT133 Copyright © 2023 Singapore University of Social Sciences (SUSS)

(Full marks: 100)

Question 1

Question 1a

Give the output produced by the sequence of statements below:

```
maximum = 30
outcome = 0
for i in range(3, 15, 3):
    print(outcome, end=" ")
    maximum = maximum + outcome
    outcome = outcome - i

print()
print(maximum)
```

(6 marks)

Question 1b

Study the given *main()* function below:

```
def main():
    num = int(input("Enter your lucky number: "))

    if num % 3 == 0 and num > 3:
        print("Is a good day!!")
    elif num == 7:
        print("Lucky 7")
    elif num <= 3 or num > 9:
        print("Here or there??")
    else:
        print("Cool....")

main()
```

When the *main()* function executes, it will ask for an integer number ("lucky number").

```
Enter your lucky number:
```

Provide **TWO (2)** inputs for the "lucky number", such that the *main()* function prints "Cool…."?

(6 marks)

Question 1c

In the *main()* function below, complete the 4 calls to the *testSelection()* function so that the output of the program is:

C1

C2

C3

C4

```python
def testSelection(x,y,z):
    if x > y and not x > z:
        print("C1")
    elif not x >= y:
        print("C2")
    else:
        if x > y or x >= z:
            print("C3")
        else:
            print("C4")

def main():

    testSelection( ?, ?, ? )
    testSelection( ?, ?, ? )
    testSelection( ?, ?, ? )
    testSelection( ?, ?, ? )

main()
```

(8 marks)

Question 2

Read the entire question before attempting. You need to submit each part individually.

An anagram is a word formed by rearranging the letters of a different word, typically using all the original letters exactly once.

Some examples:

"dog" = "god"
"Evil" = "vile"
"cheater" = "teacher"
"Santa" = "satan"

Question 2a

Create a Python function that to check if the given 2 words are anagram.

isAnagram (word1: String, word2: String): bool
- This function returns True if both words have the same letters (ignoring case). Otherwise, return False.
- For example,
    - isAnagram (“santa”, “Satan”) returns True
    - isAnagram (“santa”, “satay”) returns False

(10 marks)

Question 2b

Making use of the function in Q2(a), write a Python program that will prompt the user to enter 2 words, and display message indicating if the 2nd word is an “Anagram” of the 1st word.

Your program should repeatedly ask the user for words, until the user enters “X” or “x” as the 1st word.

Sample program run (for reference only):

Enter 1st word: **dot**
Enter 2nd word: **got**
got is NOT an anagram of dot

Enter 1st word: **Teacher**
Enter 2nd word: **Cheater**
Cheater is an anagram of Teacher

Enter 1st word: **X**
bye

(10 marks)

Question 3

Question 3a

What is the output produced when the following *main()* function is executed?

```
1  def test1(theList):
2      myList = theList
3      theList.append(max(myList))
4      for i in range(len(theList)):
5          myList[i] = myList[i] + i
6
7  def main():
8      myList = [6,5,4]
9      test1(myList)
10     print("myList:", myList)
11
12 main()
```

(9 marks)

Question 3b

What is the output produced when the following *main()* function is executed?

```
1  def test2(theList, theTuple):
2      myList = list(theTuple)
3      if len(myList) > 0:
4          theList = myList
5          print("Line 5:", theList, theTuple)
6          myList[0] = theList[0] + 10
7          print("Line 7:", theList, theTuple)
8
9  def main():
10     myList = [1,2,3]
11     myTuple = (3,4,5)
12     print("Line 12:", myList, myTuple)
13     test2(myList, myTuple)
14     print("Line 14:", myList, myTuple)
15
16 main()
```

(9 marks)

Question 3c

Given a list of words, write a program that prompts for 2 input - length of a word and a vowel. Your program will search through the list of words and displays all the words that matches the length and contains the vowel. Assume all input values are correct.

Start the program as follows:

```
def main():
    wordlist = ['python', 'java', 'pickle', 'bola', 'tournament']
    length = int(input('Enter length of word: '))
    vowel = input('Enter vowel in word: ')

    # continue your code here…
```

For the above list, if input length is "4" and the vowel is letter "a", the following would be displayed:

java
bola

If no words match the criteria, display "No words".

(12 marks)

Question 4

In Singapore, a Certificate of Entitlement (COE) gives you the right to own and use a vehicle in Singapore. To register a vehicle, you must first bid for a COE. You can bid for a COE during the open bidding exercises conducted twice a month. When you secure a COE from the bidding exercise, you can register a vehicle and use it for 10 years.

At each bidding exercise, bidder indicates their ID (either NRIC or FIN) and the reserve price, i.e., the maximum amount bidder is willing to pay for the COE. This information is stored in text files. Each line contains the ID and the reserve price, separated by comma.

Below is a sample of a "May2023-CatA.txt" file:

```
T6007603G, 90001
F1234567D, 91000
S3942244A, 92000
G1234567D, 91000
M1234567D, 91900
S8742909A, 90002
S9211367A, 92000
S0945608B, 91000
```

Write Python functions, such that the following Python application can execute.

```
def main():
    biddingData = readBiddingData("May2023-CatA.txt")
    successfulBids = getSuccessfulBids(biddingData, 5)
    print(successfulBids)
    writeBiddingResult("May2023-CatA-Result.txt", successfulBids)
```

Question 4a

readBiddingData (filename: String): dictionary
- Given a filename as parameter, the function reads the bidding information from the file and return a dictionary, with reserve price as the key and the list of bidder IDs as the value.

The populated dictionary from the sample file should be as follows:

```
biddingData = {
                90001: ["T6007603G"],
                91000: ["F1234567D","G1234567D","S0945608B"],
                92000: ["S3942244A","S9211367A"],
                91900: ["M1234567D"],
                90002: ["S8742909A"]
                }
```

(10 marks)

Question 4b

getSuccessfulBids (biddingData: dictionary, quota: int): list
- This function has 2 parameters:
  (1) biddingData: dictionary populated with bidding information, from function *readBiddingData()*
  (2) quota: the number of COE available for this bidding exercise
- Using "quota", the function will start allocating COE to bidders with highest reserve price.
- If the quota is not exhausted, the function will look into the next higher reserve price, and so on.
- At the end of the allocation exercise, this function should return the list of successful bidders' ID in a list.

With the biddingData and COE quota of 5, this function should be returning the following list:

```
["S3942244A","S9211367A","M1234567D","F1234567D","G1234567D"]
```

Note that the last bidder in reserve price 91,000 is not successful, as the quota has exhausted.

(15 marks)

Question 4c

writeBiddingResult (filename: String, result: list)

- Given the list of successful bidders' ID in parameter "result", this function writes the result to filename provided.

- Each line will contain the bidder's ID.

Sample output file:

```
S3942244A
S9211367A
M1234567D
F1234567D
G1234567D
```

(5 marks)

Appendix:

Python Built-in functions:

Function	Description
enumerate()	Takes a collection (e.g. a tuple) and returns it as an enumerate object
float()	Returns a floating point number
format()	Formats a specified value
input()	Allowing user input
int()	Returns an integer number
max()	Returns the largest item in an iterable
min()	Returns the smallest item in an iterable
open()	Opens a file and returns a file object
print()	Prints to the standard output device
range()	Returns a sequence of numbers, starting from 0 and increments by 1 (by default)
round()	Rounds a number
sorted()	Returns a sorted list
sum()	Sums the items of an iterator
zip()	Returns an iterator, from two or more iterators

Selection of List, Dictionary and File methods:

Method	Description
<dict>.keys()	Returns a sequence of keys
<dict>.values()	Returns a sequence of values
<dict>.items()	Returns a sequence of tuples (key, value) representing the key-value pairs
<key> in <dict>	Returns true if dictionary contains the specified key, false if it doesn't
for <var> in <dict>:	Loop over the keys
<dict>.get(<key>, <default>)	If dictionary has key, returns its value; otherwise returns default
del <dict>[<key>]	Deletes the specified entry
<dict>.clear()	Deletes all entries

<file>.read()	Returns the unread content as a single string
<file>.readline()	Returns the next line of the file
<file>.readlines()	Returns a sequence (a list) of unread lines in the file
<file>.write(str)	Writes string to the file, and return the number of characters
<file>.close()	Closes file and release resources
open(filename, filemode)	Opens file
<list>.append(item)	Adds item at end of list
<list>.insert(pos, item)	Adds item at specified position of list
<list>[pos] = value	Replaces element at pos with value
<list>[start:end] = sequence	Replaces elements at pos start to end -1 with elements in sequence
<list>.remove(item)	Removes item in list
<list>.pop(pos)	Removes item at pos in list
<list>.clear()	Removes all items in list
list(sequence)	Converts sequence to list
<list>.sort()	Sorts (order) the list. A comparison function may be passed as a parameter
<list>.reverse()	Reverses the list
<list>.index(x)	Returns index of first occurrence of x
<list>.count(x)	Returns the number of occurrences of x in list

----- END OF PAPER -----